

Welcome to Chapter 2. In this session, we are going to dive deep into the HTTP Protocol, also known as the Hyper Text Transfer Protocol. HTTP is the absolute foundation of the World Wide Web. It defines the rules for how data is exchanged between a client, like your web browser, and a server.

Let us start with the basic concepts of HTTP. First of all, HTTP is a text based protocol. It traditionally operates on top of TCP, which belongs to the Transport Layer. However, you should note that HTTP 3 has shifted towards using UDP and the QUIC protocol. To understand HTTP, we must understand TCP. Its core features include reliability, which ensures data integrity. It uses a three way handshake to confirm that both sides can send and receive. It also guarantees ordered delivery and provides flow control to prevent buffer overflows.

Next, we look at the evolution of HTTP. In 1991, Tim Berners Lee invented HTTP 0.9 for cross platform document sharing. By 1996, HTTP 1.0 introduced headers and status codes. In 1997, HTTP 1.1 became the most widely used version, bringing in the keep alive mechanism for persistent connections. In 2015, HTTP 2 made a huge leap in performance with multiplexing. Finally, in 2022, HTTP 3 was released using QUIC over UDP to reduce latency and improve stability during network switches.

A key mechanism we mentioned is Keep Alive. In the old HTTP 1.0, a new TCP connection was established for every single file request and closed immediately after. Since HTTP 1.1, persistent connections are enabled by default. This allows multiple requests and responses to share the same TCP connection, which significantly reduces the latency caused by repeated three way handshakes.

Now, let us examine HTTP Methods. These define the type of action you want to perform. GET is used to request resources and read data. POST is used to submit data for creating or modifying resources. PUT is for updating records. DELETE is for removing them. HEAD only returns the

headers without the body. And PATCH is used for making partial modifications to data.

Understanding the lifecycle of an HTTP request is crucial. It begins with a DNS query to translate a domain name into an IP address. Then a TCP connection is established via a three way handshake. For HTTPS, a TLS handshake follows for key exchange. Next, the client sends the actual request data. The server then processes this request, perhaps running code or querying a database. Finally, the server sends back the response data, and the browser renders the results using its HTML, CSS, and JavaScript engines.

Security is a top priority on the web. While standard HTTP is unencrypted and vulnerable to eavesdropping, HTTPS uses TLS or SSL to encrypt data. This relies on digital certificates issued by a Certificate Authority, which act as a server ID and contain a public key. This ensures privacy by turning data into unreadable text and guarantees integrity by using hashing and digital signatures to prevent tampering.

Let us break down the structure of a URL. In this example, HTTPS is the protocol. The domain name is the host. Port 443 is specified for secure connections. The path points to the specific resource, in this case, a person with ID 123. Finally, the query parameters after the question mark provide additional data, like the name John and age 25.

Here is an example of an actual HTTP Request. It starts with the method GET, followed by the path and the protocol version HTTP 1.1. Below that are the headers, such as the Host domain, the User Agent which identifies the browser, and the Accept header which tells the server what kind of data the client can handle.

And here is the corresponding HTTP Response. It starts with status code 200 OK. Important headers include Content Type, which describes the data nature like HTML or an image, and Content Length, which specifies the size of the response body. The connection is marked as keep alive, and finally, the actual HTML content follows.

HTTP uses status codes to communicate the result. 100 level codes are informational. 200 level codes mean success, such as 200 OK. 300 level codes are for redirection, like 301 Moved Permanently. 400 level codes indicate client errors, like the famous 404 Not Found. 500 level codes mean the server encountered an error, such as 500 Internal Error.

Now it is time for some hands on practice with browser DevTools. The Elements panel shows you the HTML structure and CSS styles. The Console is for debugging JavaScript. Most importantly for us, the Network panel records all data transmissions. You should pay attention to the Headers for metadata, the Payload for submitted data, and the Preview or Response tabs to see what the server actually sent back.

Within the Network panel, you can see the Timing waterfall. This is divided into five stages. Queueing is when the request is waiting for a connection. Stalled is when it is waiting for a socket. The DNS, TCP, and SSL stages represent the connection overhead. TTFB, or Time to First Byte, is the most critical performance metric, representing the server processing time. Finally, Content Download is the time taken to receive the data.

You can also master HTTP using the terminal tool called curl. On Windows, you can use the command prompt or PowerShell. On macOS, open the Terminal via Spotlight. On Linux, use Ctrl Alt T. Try using curl dash capital I to see only the headers of a website, or curl dash v for verbose mode, which shows the entire process including the TCP handshake.

That concludes our overview of the HTTP protocol. Thank you for your attention. Are you prepared to start analyzing network packets now?